影像處理
--CH3_3

姓名:邱榮誠

學號:314512065

系級:電控碩 1

日期:2025/10/13

Q1.Please use a self-designed Lowpass Gaussian Filter Kernels to cancel the shaded pattern noise of the image, 'checkerboard1024- shaded.tif'. Please describe the your lowpass Gaussian filter kernels, shading noise pattern as Fig. 3.42 (b), final corrected image without shading noise pattern as Fig. 3.42 (c) and print out the source code?

1）**source code**：

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

def gaussian_kernel(ker_size: int, sigma: float) -> np.ndarray:
    c = ker_size // 2
    y, x = np.ogrid[:ker_size, :ker_size]
    g = np.exp(-((x - c) ** 2 + (y - c) ** 2) / (2.0 * sigma *
sigma)).astype(np.float32)
    g /= g.sum()
    return g

if __name__ == "__main__":
    img = cv2.imread("checkerboard1024-shaded.tif",
cv2.IMREAD_GRAYSCALE)
    img_f = img.astype(np.float32)
    ksize = 256.0
    sigma = 64.0
    kernel = gaussian_kernel(ksize, sigma)
    shading = cv2.filter2D(img_f, ddepth=-1, kernel=kernel,
borderType=cv2.BORDER_CONSTANT)
    eps = 1e-6
    shading_safe = np.maximum(shading, eps)
    corrected = (img_f / shading_safe) * 255.0
    corrected = np.clip(corrected, 0, 255).astype(np.uint8)
    cv2.imwrite("original.png", img)
    cv2.imwrite("shading.png", shading.astype(np.uint8))
    cv2.imwrite("corrected.png", corrected)
    plt.figure(figsize=(9, 4))
```

```
    plt.subplot(1, 3, 1); plt.imshow(img,
cmap="gray");       plt.title("Original");  plt.axis("on")
    plt.subplot(1, 3, 2); plt.imshow(shading,
cmap="gray");   plt.title("Shading (LPF)"); plt.axis("on")
    plt.subplot(1, 3, 3); plt.imshow(corrected, cmap="gray");
plt.title("Corrected"); plt.axis("on")
    plt.tight_layout(); plt.show()
```

2)describe the lowpass Gaussian filter kernels

因這張圖與課本Figure3.42相同，因此參照了書上的方法。先求出checkerboard1024每個方格的大小為$\frac{1024}{16}$ x $\frac{1024}{16}$ =64x64，所以設計Gaussian kernel size為每個方格大小的四倍為256x256，σ等於每個方格的大小為64。

Consider the 2048×2048 checkerboard image in Fig. 3.42(a), whose inner squares are of size 128×128 pixels. Figure 3.42(b) is the result of lowpass filtering the image with a 512×512 Gaussian kernel (four times the size of the squares), $K = 1$, and $\sigma = 128$ (equal to the size of the squares). This kernel is just large enough to blur-out the squares (a kernel three times the size of the squares is too small to blur them out sufficiently). This result is a good approximation to the shading pattern visible in Fig. 3.42(a). Finally, Fig. 3.42(c) is the result of dividing (a) by (b). Although the result is not perfectly flat, it definitely is an improvement over the shaded image.

Figure1.the method in testbooks

3)describe the shading noise pattern

此圖為對原始影像光照場的估計，用以呈現環境光照的分布情形。由於原圖中光照特性為邊角較暗、中央較亮，經由 Gaussian 低通濾波後，高頻的棋盤格細節被平滑抑制，僅保留低頻的光照變化，因此清楚顯示出光源造成的陰影梯度分布。

4)describe the corrected image without shading noise pattern:

校正後的影像是將原始含有陰影的棋盤格影像除以估計出的光照場，並重新縮放強度而得到的。經過校正後，原本因光照不均造成的邊角變暗現象已經消除，整張棋盤格的亮度與對比度在全域上變得均勻一致。這代表 Gaussian 低通濾波法成功地將高頻結構細節（棋盤格紋）與低頻陰影雜訊分離，得到一幅不再受光照不均影響的正確影像。

5)checkerboard1024 result:

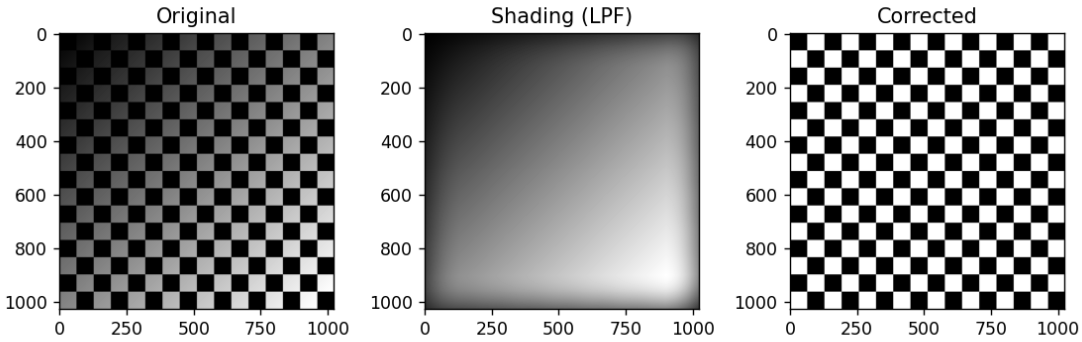

Figure2. checkerboard1024 result

Q2.Repeat (1) steps in the image 'N1.bmp'

1）**N1.bmp source code**:

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

def gaussian_kernel(ksize: int, sigma: float) -> np.ndarray:
    assert ksize % 2 == 1 and ksize > 1
    c = ksize // 2
    y, x = np.ogrid[-c:c+1, -c:c+1]
    g = np.exp(-((x**2 + y**2) / (2.0 * sigma *
sigma))).astype(np.float32)
    g /= g.sum()
    return g

def remove_shading_lpf(img_gray: np.ndarray, ksize: int = 121, sigma:
float = 20.0, floor_percentile: float = 5.0) -> tuple[np.ndarray,
np.ndarray]:
    img_f = img_gray.astype(np.float32)
    kernel = gaussian_kernel(ksize, sigma)
    shading = cv2.filter2D(img_f, ddepth=-1,
kernel=kernel,borderType=cv2.BORDER_REFLECT101)
    floor = float(np.percentile(shading, floor_percentile))
```

```
    shading_safe = np.maximum(shading, floor)
    gain = float(np.median(shading_safe))
    corrected = img_f * (gain / shading_safe)
    corrected = np.clip(corrected, 0, 255).astype(np.uint8)
    return shading, corrected

if __name__ == "__main__":
    img = cv2.imread("N1.bmp", cv2.IMREAD_GRAYSCALE)
    KSIZE = 121
    SIGMA = 20.0
    shading, corrected = remove_shading_lpf(img, ksize=KSIZE,
sigma=SIGMA, floor_percentile=5.0)
    cv2.imwrite("original.png", img)
    shading_vis = cv2.normalize(shading, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
    cv2.imwrite("shading.png", shading_vis)
    cv2.imwrite("corrected.png", corrected)
    plt.figure(figsize=(10, 4))
    plt.subplot(1, 3, 1); plt.imshow(img,
cmap="gray");       plt.title("Original");        plt.axis("on")
    plt.subplot(1, 3, 2); plt.imshow(shading,
cmap="gray");   plt.title("Shading (LPF)");   plt.axis("on")
    plt.subplot(1, 3, 3); plt.imshow(corrected, cmap="gray");
plt.title("Corrected");       plt.axis("on")
    plt.tight_layout(); plt.show()

    ker = gaussian_kernel(KSIZE, SIGMA)
    print(f"[Kernel] size={KSIZE}x{KSIZE}, sigma={SIGMA}")
    print(f"        L1 sum={ker.sum():.6f}, center={ker[KSIZE//2,
KSIZE//2]:.6e}")
    print(f"[Shading] min={shading.min():.2f}, max={shading.max():.2f},
median={np.median(shading):.2f}")
```

2)describe the lowpass Gaussian filter kernels

考量到實驗影像N1.bmp的解析度為640×480，若以上圖checkerboard1024-shaded.tif的影像大小1024×1024為參考基準，則影像寬度比例約為：

$\frac{480}{1024}$=0.46875倍，且其kernel size為256x256。因此依照影像比例換算，N1.bmp

所需的濾波器大小可估為：256×0.46875=120，採用kernel size為121x121來確保濾波器在中心點對稱並具備奇數尺寸。此外，依高斯分佈特性，濾波器的有效範圍約為 ±3σ，因此標準差σ通常可設為濾波器大小的1/6，約為20。

3)describe the shading noise pattern

在原始影像N1.bmp中，可清楚觀察到一種沿斜向分佈的亮度不均現象。經由低通高斯濾波器平滑處理後，影像中的細節與高頻紋理被有效去除，只保留整體緩慢變化的亮度分佈。此時可明顯看出原始影像中光照較弱的暗沉區域仍然存在，呈現出整體光照場的分佈特性。

4)describe the corrected image without shading noise pattern:

在校正後的影像中，原本斜向分佈的亮度不均現象已被有效消除，整體畫面呈現出均勻且穩定的亮度分佈，物體紋理與細節亦被清晰保留。與原始影像相比，經過平場校正後的結果不再受到光照變化的干擾，且其背景灰階一致。

5) N1.bmp result:

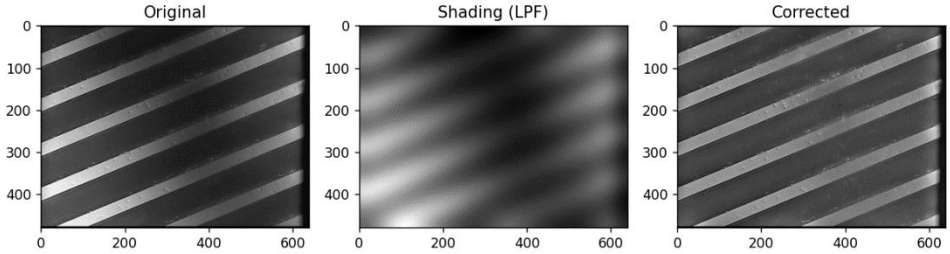

Figure3. N1.bmp result

Q3.Please comment and compare your two self-designed Lowpass Gaussian Filter Kernels?

對於 checkerboard1024-shaded.tif，其解析度較高 1024×1024，且棋盤格紋理間距明顯，因此設計的濾波器尺寸較大，採用 Kernel Size=257×257、σ=64。這樣可以涵蓋較多棋盤格單元的範圍、保留全域的光照變化。由 Figure2 結果上來看，校正過後的圖片細節更加清楚，亮暗對比分明。

對於 N1.bmp 的影像解析度僅為 640×480，影像內容細節較小、陰影變化範圍較窄。為保持相似的比例關係，本實驗依據影像尺寸比例縮小濾波器大小，設計為 Kernel Size=121×121、σ=20。此組合可在較小範圍內達到適度平滑效果，成功去除影像中沿斜向分佈的亮度不均，同時保留原物體邊緣與紋理特

微，但相較之下，由 Figure3 得知校正過後的圖片背景灰階一致，所以亮暗對比不明顯。